

- [CUDA Path Tracer](#)
 - [Part 1 - Core Path Tracer](#)
 - [Diffuse BSDF and Multi-Bounce Path Tracing](#)
 - [Early Multi-Bounce Result](#)
 - [Stream Compaction](#)
 - [Active Rays per Bounce: Open vs. Closed Scene](#)
 - [Stream Compaction Performance](#)
 - [Analysis](#)
 - [Material Sorting](#)
 - [Material Sorting Performance](#)
 - [Analysis](#)
 - [Stochastic Sampled Antialiasing](#)
 - [Sampling Pattern](#)
 - [Visual Comparison](#)
 - [Analysis](#)

CUDA Path Tracer

University of Pennsylvania, CIS 565: GPU Programming and Architecture, Project 3

- Yunzhe Deng
 - [LinkedIn](#), [personal website](#)
- Tested on: Windows 11, Intel Core i7-10750H @ 2.60GHz, 16 GB RAM, NVIDIA GeForce RTX 2060 (Personal Computer)

Part 1 - Core Path Tracer

The core renderer is a CUDA-based Monte Carlo path tracer supporting cosine-weighted diffuse scattering, multi-bounce indirect illumination, emissive surfaces, stream compaction, material sorting, and stochastic sampled antialiasing.

The primary ray is produced for each pixel starting from the camera. Based on the material and the outcome of the ray-scene intersection, the light ray terminates or continues to scatter. The above procedure repeats itself until the ray intersects with an emissive surface, exits the scene, or achieves its maximum depth of bounces.

Diffuse BSDF and Multi-Bounce Path Tracing

For diffuse surfaces, I implemented cosine-weighted hemisphere sampling using the provided `calculateRandomDirectionInHemisphere()` function. At every diffuse intersection, the current path throughput is multiplied by the material color:

```
pathSegment.color *= material.color;
```

The ray origin is moved to the current surface intersection and a new direction is randomly sampled in the hemisphere around the surface normal. Conceptually, each path follows:



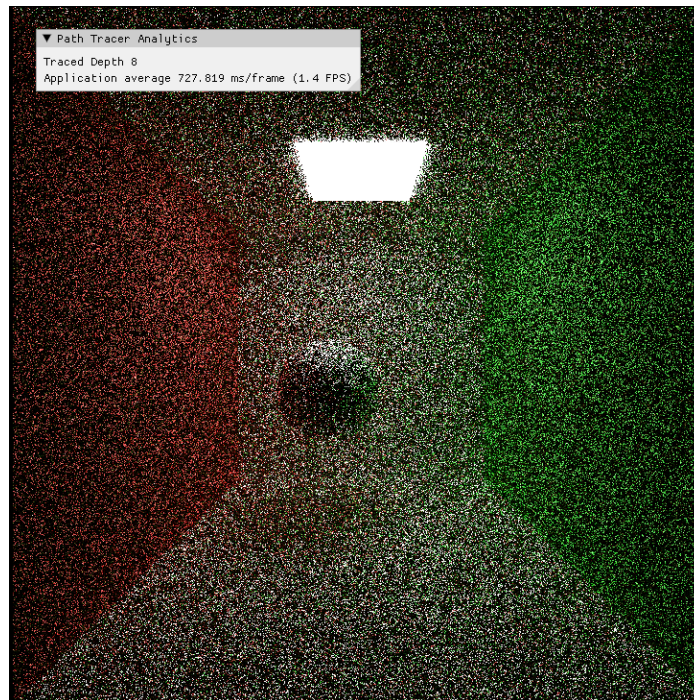
When a path reaches an emissive material, its accumulated throughput is multiplied by the light color and emittance:

```
path.color *= material.color * material.emittance;
```

The path then terminates. If a ray leaves the scene without reaching a light source, its contribution is set to zero. The renderer therefore supports indirect illumination through multiple diffuse bounces. This produces effects such as the red and green color bleeding visible in the Cornell box.

Early Multi-Bounce Result

The image below shows an early working result shortly after multi-bounce diffuse path tracing was enabled. At this point, the image is still dominated by Monte Carlo noise, but indirect illumination and color bleeding are already visible.



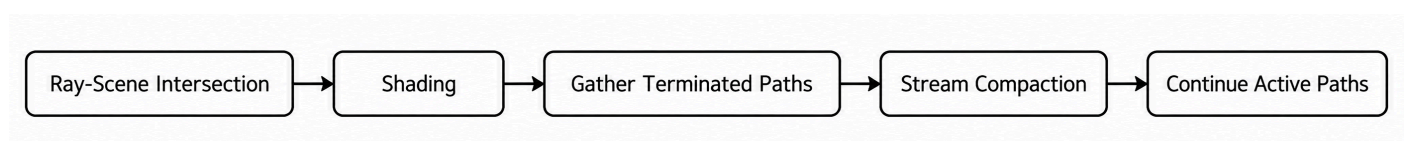
Stream Compaction

Many rays that bounce around the scene have paths that end before they get to the trace depth limit.

A path terminates when:

- it reaches an emissive surface,
- it escapes the scene, or
- it reaches the maximum allowed bounce depth.

Processing terminated rays is wasteful since they cannot contribute to any image generation. Therefore, stream compaction is done after each bounce. The rendering loop follows the structure below:



The contribution of the final color from each terminated path to the image buffer is made by adding it to its respective `pixelIndex`. The terminated paths are deleted from the list of active paths. The total count of active paths is obtained from the new end of the path buffer:

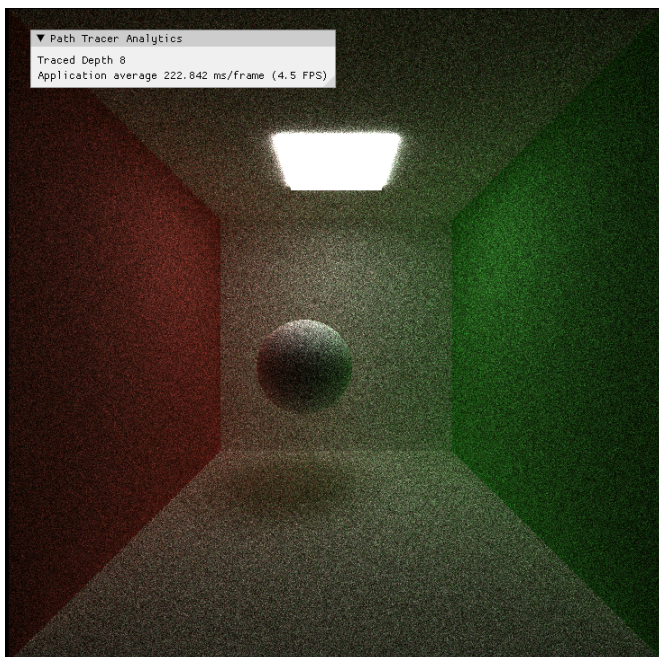
```
num_paths = static_cast<int>(dev_path_end - dev_paths);
```

The rendering loop continues until no active paths remain.

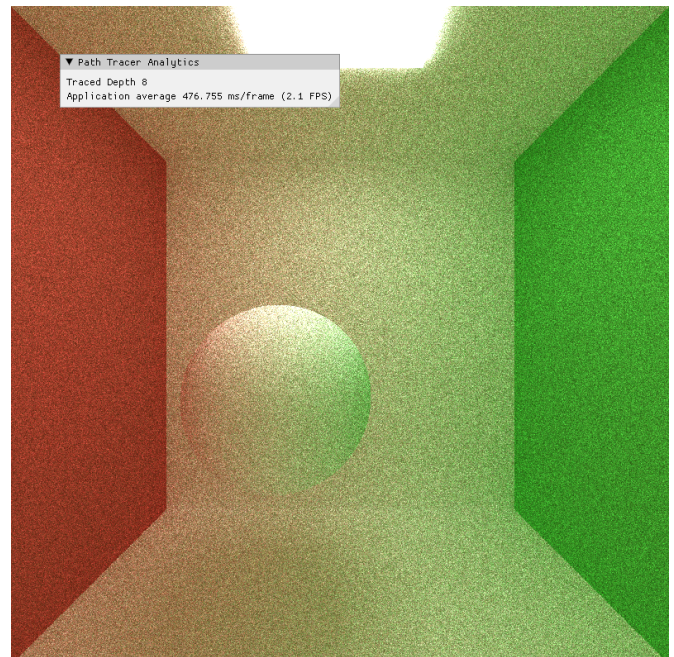
Active Rays per Bounce: Open vs. Closed Scene

The effectiveness of stream compaction depends strongly on scene geometry. In an open scene, rays can escape the scene and terminate early. In a closed scene, rays are surrounded by geometry and are more likely to remain active for additional bounces (for the closed scene, I added a front wall at $z = +5$, then moved the camera inside the box so that rays could no longer escape through the open front, this modified scene is saved as `scenes/cornell_closed.json`). The contrast of two scenes is shown below.

Open Scene

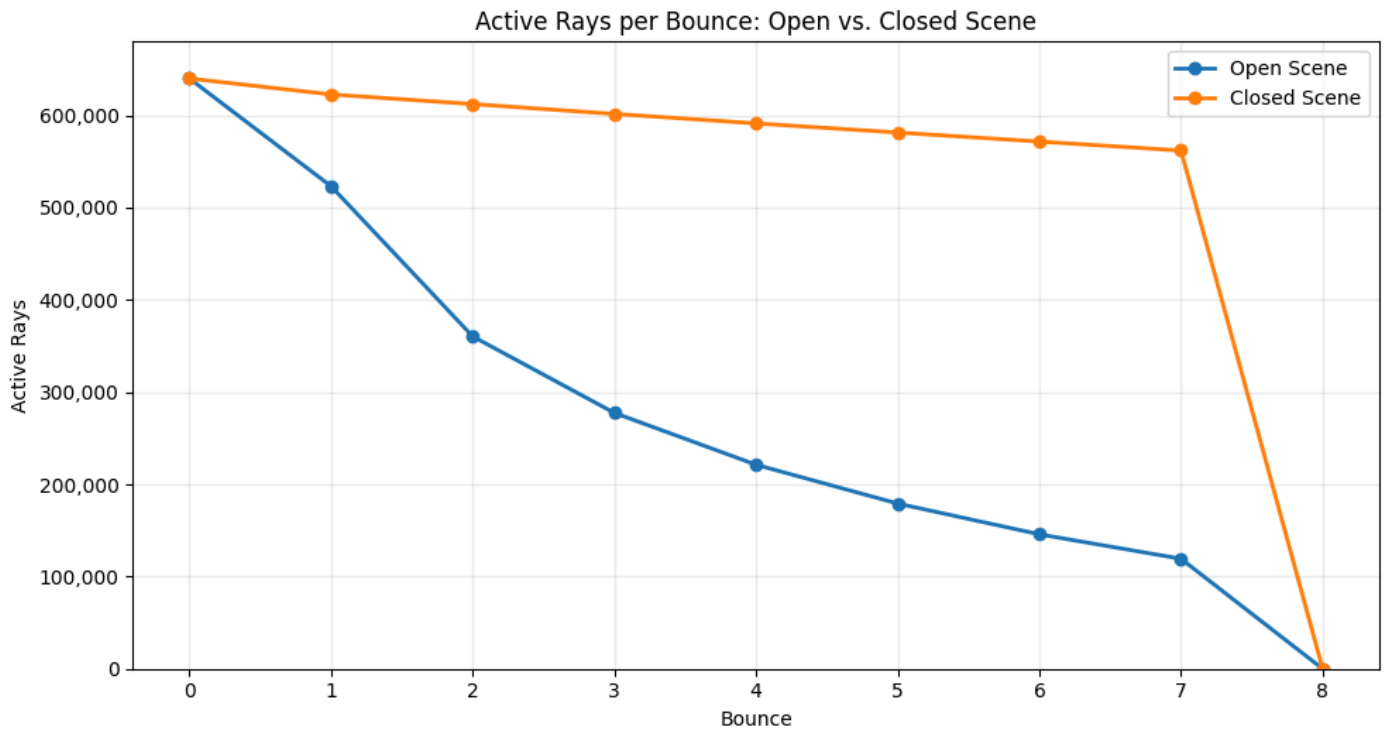


Closed Scene



The number of active rays after each bounce is shown below. The effectiveness of stream compaction depends strongly on scene geometry. In an open scene, rays can escape the scene and terminate early. In a closed scene, rays are surrounded by geometry and are more likely to remain active for additional bounces. To create the closed-scene test, I duplicated the original Cornell box scene, added a front wall at $z = +5$, and moved the camera inside the box so that rays could no longer escape through the open front. This modified scene is saved as `scenes/cornell_closed.json`.

Scene / Bounce	0	1	2	3	4	5	6	7	8
Open Scene	640000	523164	360348	277462	221273	179171	145828	119308	0
Closed Scene	640000	622757	612276	601630	591352	581445	571626	561930	0



The difference is significant. By bounce 7, the open scene contains only 119,308 active rays, while the closed scene still contains 561,930 active rays. In the open scene, there would be many rays that escape and stop before getting to the end point; therefore, this will allow for the elimination of lots of unnecessary processing via stream compaction. In the closed scene, there would be many rays bouncing off surfaces and hence lots of active paths. This is why stream compaction offers greater potential for saving unnecessary processing in an open scene.

Stream Compaction Performance

All performance measurements were collected using the Release build with the same resolution, maximum trace depth, and scene configuration for each comparison.

Scene	Stream Compaction	Time / Frame	FPS
Open	OFF	84.298 ms	11.9
Open	ON	40.616 ms	24.6

Scene	Stream Compaction	Time / Frame	FPS
Closed	OFF	102.587 ms	9.7
Closed	ON	103.924 ms	9.6

Analysis

Stream compaction offers considerable performance gain in the open scene. The average frame time decreases from **84.298 ms** to **40.616 ms**, that is, by about **51.8%**, and the frame rate grows from **11.9 FPS** to **24.6 FPS**. It implies **2.08× speedup**.

This outcome is fully consistent with the active-ray measurements above. In the open scene, numerous rays escape or get terminated within the first several bounces so that the number of active paths drops from 640,000 primary rays to only 119,308 by bounce 7. Stream compaction discards these terminated paths, enabling the intersection and shading kernels to process a substantially reduced amount of data.

On the other hand, there is no performance gain achieved in the closed scene through stream compaction. The frame time rises slightly from **102.587 ms** to **103.924 ms**, i.e., about **1.3%**, while the frame rate stays almost the same at **9.7 FPS** and **9.6 FPS**, respectively. In the closed scene, the rays cannot escape easily and 561,930 out of 640,000 initial rays remain active at bounce 7. As a consequence, not a lot of paths can be discarded, making the reduction of GPU work negligible compared to the cost of performing stream compaction.

These results demonstrate that the effectiveness of stream compaction depends strongly on how quickly paths terminate. It is highly beneficial for open scenes with substantial early ray termination, but can introduce unnecessary overhead in closed scenes where most rays remain active until the maximum path depth.

Material Sorting

Different material types may require different BSDF calculations. When neighboring GPU threads evaluate different material branches, warp divergence can reduce shading efficiency. To improve shading coherence, I implemented material-based path sorting before the shading stage. After ray-scene intersection, each active path receives a sorting key corresponding to the `materialId` of its intersection. The rendering pipeline becomes:



The material IDs are used as sorting keys while the corresponding `PathSegment` and `ShadeableIntersection` data remain paired during sorting. Paths that interact with the same material are stored contiguously in memory after sorting before shading. Material sorting can be disabled/enabled for direct comparison of performance difference due to it.

Material Sorting Performance

Configuration	Time / Frame	FPS
Material Sorting OFF	37.128 ms	26.9
Material Sorting ON	73.836ms	13.5

Analysis

Material sorting did not improve performance for the current Cornell box scene. With material sorting disabled, the renderer required **37.128 ms/frame**, while enabling material sorting increased the frame time to **73.836 ms/frame**. This corresponds to an approximately **98.9% increase in frame time**, meaning that the sorted version was almost twice as slow in this test.

The primary reason is that material sorting introduces additional work at every bounce. Material keys must first be generated, and `thrust::sort_by_key` must then reorder the active paths and their corresponding intersections before shading. This sorting and memory movement adds a significant amount of overhead.

In the current renderer, the majority of non-emissive surfaces share the same diffuse BSDF. Even if the surfaces on the red, green, and white Cornell boxes are of different materials, the shading process for all of them is almost the same. Consequently, there is no sufficient material-dependent divergence caused by the shading kernel to enable the gain from sorting.

Material sorting is expected to become more useful in scenes containing a larger variety of computationally different BSDFs, such as diffuse, specular, and refractive materials. In those cases, grouping paths by material can reduce divergence more substantially and may better offset the cost of sorting.

Stochastic Sampled Antialiasing

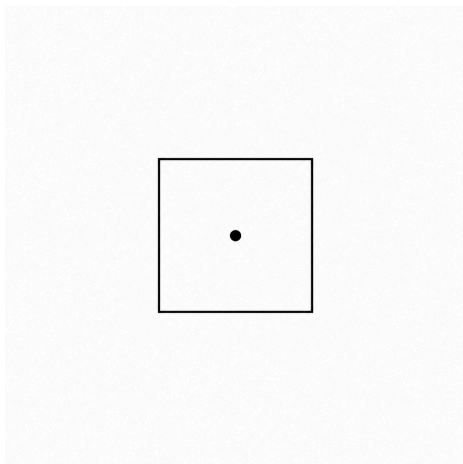
Sampling Pattern

Without stochastic antialiasing, every iteration traces the camera ray through the same fixed sub-pixel location. Repeatedly sampling the same location can produce visible aliasing along high-contrast geometry boundaries. To address this, I implemented stochastic sampled antialiasing by jittering the primary ray independently in both the x and y directions for every pixel and every iteration:

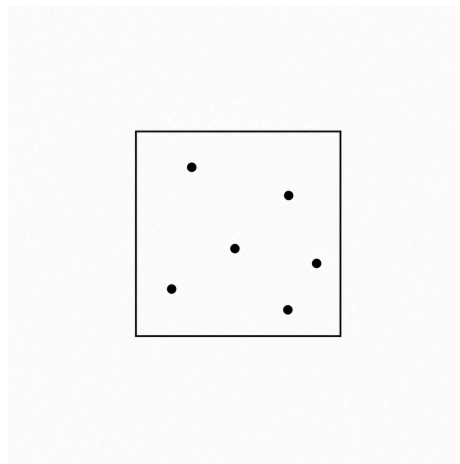
```
float jitterX = u01(rng);  
float jitterY = u01(rng);  
  
float sampleX = (float)x + jitterX;  
float sampleY = (float)y + jitterY;
```

This causes each iteration to sample a slightly different sub-pixel location. Over many iterations, these samples are averaged together, producing smoother estimates of pixel coverage and reducing jagged edges.

Single Fixed Sample per Pixel



Random Jittered Samples per Pixel

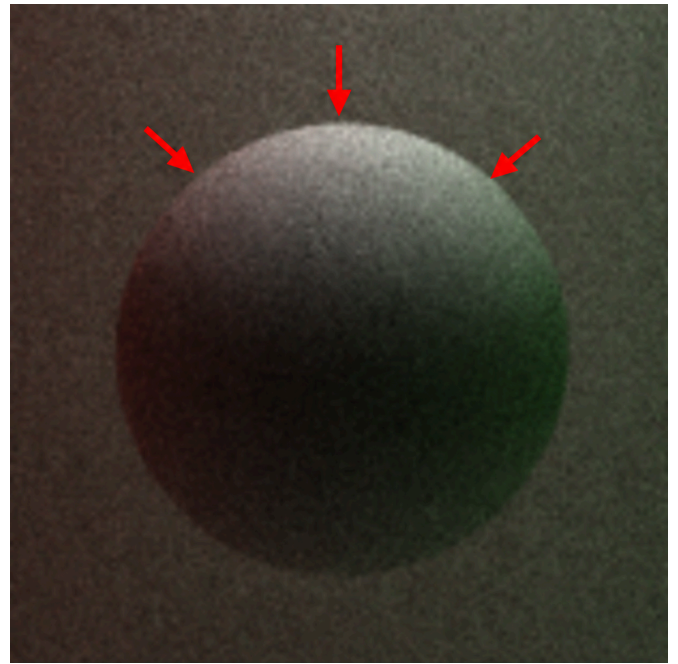
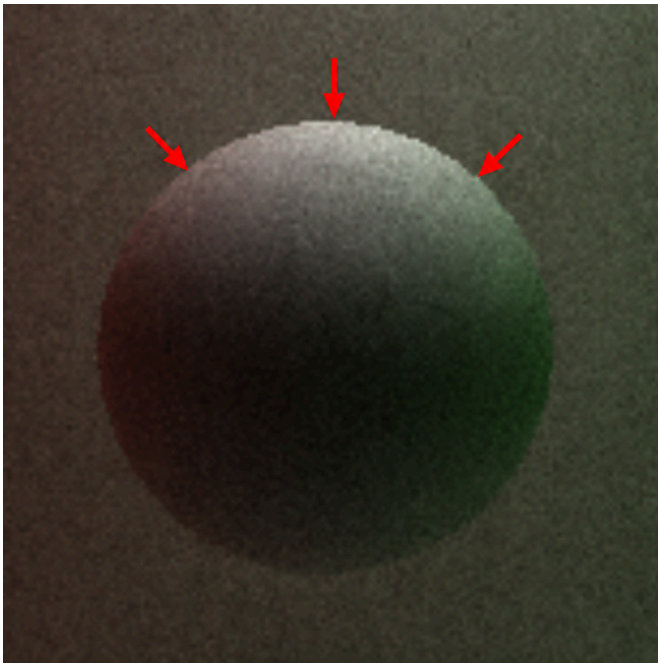


Visual Comparison

The effect of stochastic antialiasing is most visible along object silhouettes and lighting boundaries. The following comparison was rendered using the same scene and settings, with antialiasing disabled on the left and enabled on the right.

Antialiasing OFF

Stochastic Antialiasing ON



Analysis

Based on the antialiasing test, the results show that stochastic sampled antialiasing enhances the appearance of the sphere silhouette due to reduced stair-case effect on the object boundary. From the image without antialiasing, one can see that the edge of the sphere looks more pixelated since the sampling takes place from the same sub-pixel coordinates per iteration. However, when antialiasing is applied, each iteration jitters the sampled position within the pixel so as to get an accurate average approximation of partial pixel coverage.

The greatest improvement is seen on the curved outer boundary of the sphere particularly the upper left and right edges that have been marked on the comparison images. While both renderings possess Monte Carlo noise, antialiased rendering keeps the silhouette smooth. This means that stochastic sub-pixel sampling makes edge quality better.